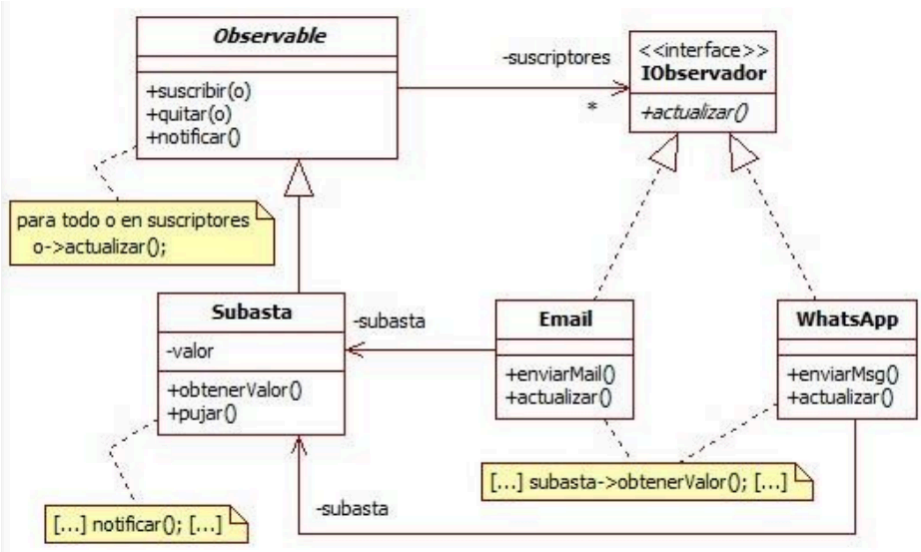
T10. Patrones de comportamiento

Observer

El patrón **Observer** (Observador) es un patrón de comportamiento cuyo objetivo es establecer una relación de dependencia uno a muchos entre objetos. Esto significa que cuando un objeto, llamado sujeto, **cambia su estado**, todos los objetos que dependen de él, llamados observadores, son **notificados automáticamente** y pueden **actualizarse** en consecuencia. De esta forma, se consigue mantener **sincronizados** distintos objetos sin que exista un acoplamiento fuerte entre ellos.

- <u>Aplicación</u>

Este patrón resulta especialmente útil cuando un cambio en un objeto implica modificar otros, **sin importar cuántos** sean ni quiénes son exactamente. Es decir, el sujeto no necesita conocer los detalles concretos de los observadores, solo sabe que implementan una interfaz común. También se utiliza cuando un objeto debe ser capaz de **enviar notificaciones** de forma genérica, sin hacer suposiciones sobre **quienes** son los receptores de esas notificaciones, lo que aporta flexibilidad y reutilización al diseño.



En la parte superior izquierda del diagrama se encuentra la clase **Observable**, que define los métodos fundamentales como suscribir(), quitar() y notificar(). Mantiene una colección de suscriptores (observadores), sobre la que itera para invocar el método actualizar() de cada uno cuando ocurre una notificación. A la derecha de Observable aparece la interfaz **IObservador**, incluye el método actualizar(), que será llamado por el sujeto cuando haya cambios. Debajo de Observable encontramos la clase **Subasta**, que actúa como **SujetoConcreto**, hereda de Observable y añade su propio estado, representado por el atributo valor.

Cuando se produce una **puja** y cambia el valor de la subasta, se invoca el método **notificar()**, que a su vez desencadena la **actualización** de todos los **observadores** suscritos. En la parte inferior derecha se encuentran las clases **Email** y **WhatsApp**, que son los ObservadoresConcretos. Ambas implementan la interfaz IObservador (el método actualizar()), cada una mantiene una referencia a la subasta lo que les permite consultar su estado cuando reciben la notificación.

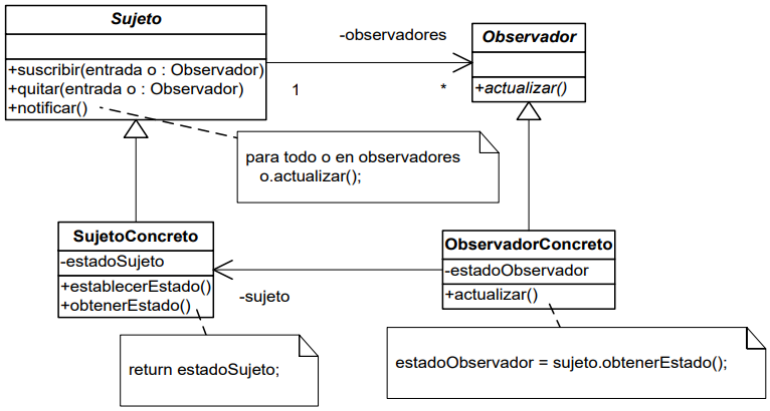
- <u>Participantes</u>

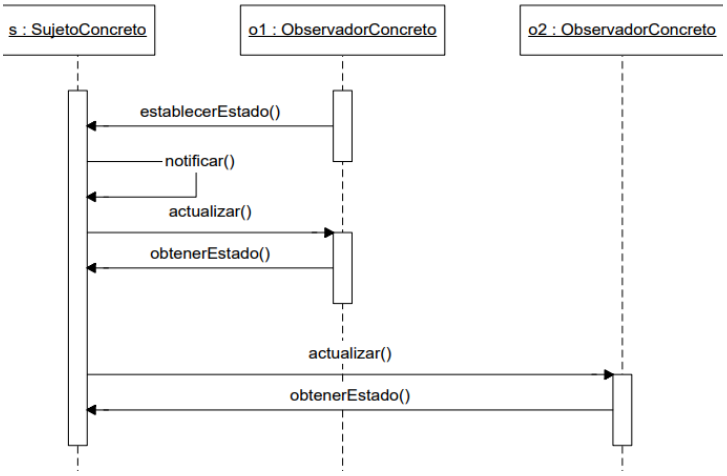
- **Sujeto** (Subject): es quien mantiene una colección de **observadores** (no conoce los detalles) y proporciona **métodos** para suscribirse, eliminarse y notificar cambios.

- **Observador** (Observer): define una **interfaz** común, normalmente con un método como actualizar(), que se invoca cuando hay cambios en el sujeto.

- **SujetoConcreto** (ConcreteSubject): almacena el **estado** que interesa a los observadores y se encarga de notificar cualquier cambio.

- **ObservadorConcreto** (ConcreteObserver): mantiene una referencia al sujeto y sincroniza su estado interno cuando recibe una notificación.





- <u>Ventajas e inconvenientes</u>

Permite **modificar** de forma **independiente** tanto el sujeto como los observadores, ya que tienen acoplamiento mínimo (el sujeto no necesita saber la clase concreta de ningún observador). Además, **facilita** la **comunicación** mediante **difusión**, permitiendo que múltiples objetos reaccionen automáticamente a un mismo cambio. Sin embargo, presenta inconvenientes: los observadores no se conocen entre sí, lo que puede provocar **actualizaciones en cascada** difíciles de controlar, y el mecanismo de **actualización** es **simple**, sin detallar exactamente qué ha cambiado en el sujeto, lo que puede obligar a consultas adicionales.

- <u>Implementación</u>

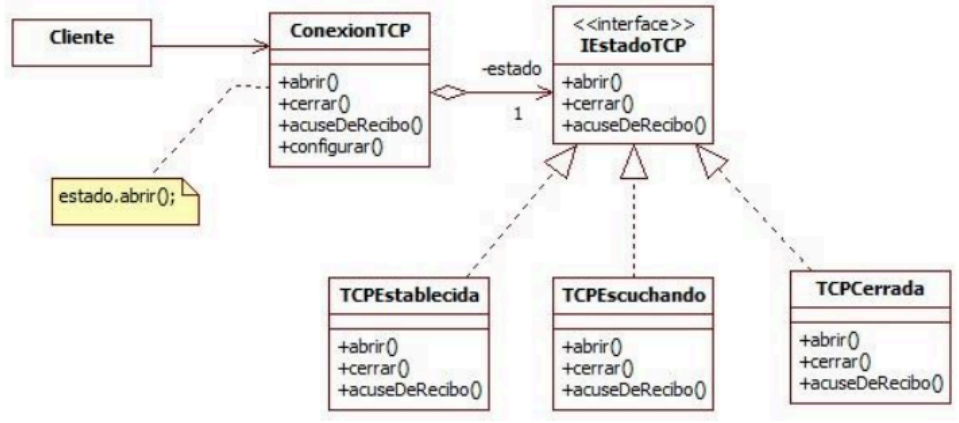
Un **observador** puede estar **suscrito** a **varios sujetos**, lo que implica que el método actualizar() debe ser lo suficientemente flexible para manejar múltiples fuentes de cambio. También es necesario definir una **política de actualización**: si las notificaciones se producen automáticamente al cambiar el estado del sujeto o si son los propios clientes quienes las desencadenan. Es fundamental documentar qué operaciones del sujeto generan notificaciones, así como decidir el **nivel de detalle** de la información proporcionada a los observadores. Por último, es importante controlar qué **eventos son relevantes** para cada observador, evitando notificaciones innecesarias y mejorando la eficiencia del sistema.

State (Estado)

El patrón **State** (Estado) es un patrón de comportamiento cuyo objetivo es permitir que un objeto **modifique su comportamiento** cuando cambia su estado interno. Como consecuencia, el objeto parece cambiar de clase en tiempo de ejecución, aunque en realidad lo que cambia es el objeto estado al que delega su comportamiento.

- <u>Aplicación</u>

Se utiliza cuando el **comportamiento** de un objeto **depende** claramente de su **estado** y este puede cambiar dinámicamente durante la ejecución del programa. También es especialmente útil cuando existen métodos con estructuras condicionales con múltiples ramas formadas por bloques extensos de código.



En la parte superior izquierda del primer diagrama se encuentra la clase **Cliente**, que interactúa directamente con la clase **ConexionTCP**. Esta clase representa el contexto y define operaciones como abrir(), cerrar(), acuseDeRecibo() y configurar(). Internamente, mantiene una **referencia** a un objeto denominado estado, que sigue la interfaz **IEstadoTCP**. A la derecha aparece la interfaz IEstadoTCP, que declara las operaciones abrir(), cerrar() y acuseDeRecibo(). Estas operaciones serán implementadas de forma distinta por cada estado concreto.

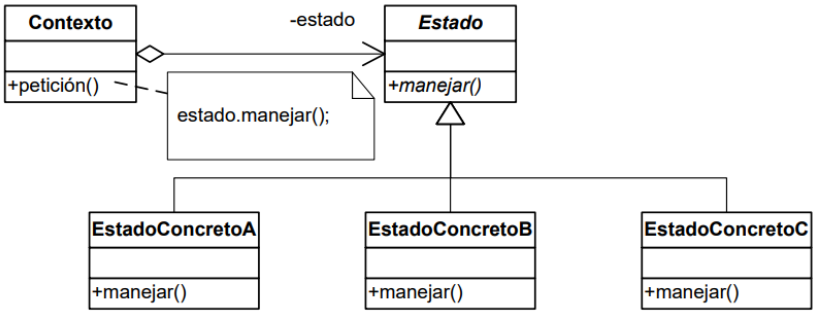
Debajo de esta interfaz se encuentran las clases **TCPEstablecida**, **TCPescuchando** y **TCPCerrada**, que representan los distintos estados posibles de la conexión. Cada una redefine el comportamiento de las operaciones según el estado en el que se encuentra la conexión.

- <u>Participantes</u>

- **Contexto** (Context): es la clase principal con la que interactúan los clientes. Define la interfaz de interés y mantiene una referencia a un objeto de tipo EstadoConcreto que representa su estado actual.

- **Estado** (State): es una interfaz o clase abstracta que declara los métodos que encapsulan el comportamiento dependiente del estado.

- **EstadosConcretos** (ConcreteState): implementa el comportamiento específico asociado a cada estado particular del contexto.



- <u>Comportamiento</u>

El Contexto **delega** las peticiones dependientes del estado en el objeto estado actual. En algunos casos, el propio estado puede necesitar acceso al contexto (por ejemplo, para cambiar el estado), por lo que el contexto puede pasarse como **parámetro**. Tanto el contexto como los propios estados concretos pueden decidir cuándo y cómo se producen las transiciones entre estados.

- <u>Ventajas e inconvenientes</u>

Agrupa en un **objeto** todo el comportamiento asociado a un estado concreto, lo que facilita la comprensión y el mantenimiento del sistema. Además, permite **añadir nuevos estados** y **transiciones** fácilmente sin modificar el código existente, favoreciendo la extensibilidad. También **elimina estructuras condicionales** complejas y hace **explícitas** las **transiciones** entre estados, reduciendo el riesgo de inconsistencias. Incluso es posible **compartir** instancias de **estados** entre varios contextos si estos **no contienen atributos** propios.

- <u>Implementación</u>

Determinar quién **controla** las **transiciones** de estado:

- Si los criterios de cambio son **simples** y **fijos**, el propio **contexto** puede encargarse de cambiar de estado.
- Si se busca **mayor flexibilidad**, cada **estado** puede ser **responsable** de decidir cuál es el siguiente estado, aunque esto incrementa el acoplamiento.

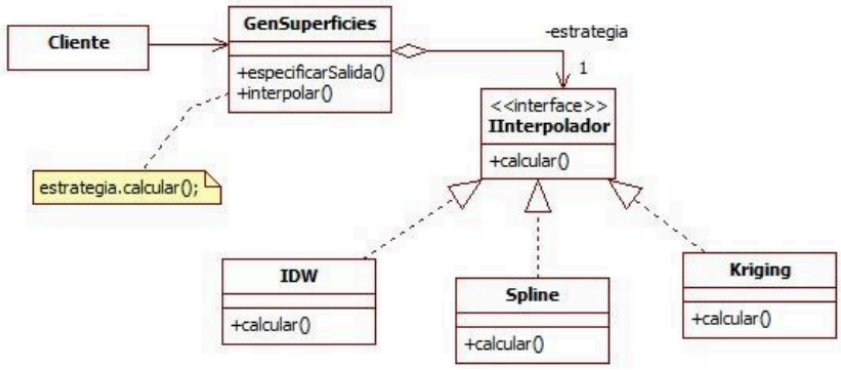
También es importante decidir cómo se gestionan las instancias de los estados: pueden crearse **bajo demanda** (solo cuando se necesitan) y **destruirse** después, o bien **crearse** todas al **inicio** y **reutilizarse** durante la ejecución.

Strategy (Estrategia)

El patrón **Strategy** (Estrategia) es un patrón de comportamiento cuyo objetivo es definir una **familia de algoritmos**, hacerlos **intercambiables** y encapsular cada uno de ellos en objetos en los que delegar peticiones. De este modo, un mismo problema puede resolverse utilizando **diferentes estrategias** sin que el cliente tenga que modificar su código. En lugar de implementar múltiples variantes de un algoritmo dentro de una misma clase mediante condicionales, cada algoritmo se encapsula en una clase independiente, permitiendo que el comportamiento pueda variar de forma dinámica.

- <u>Aplicación</u>

Este patrón se utiliza cuando existen **varias clases relacionadas** que solo difieren en su comportamiento, cuando se necesitan distintas **variantes** de un mismo **algoritmo** o cuando un algoritmo usa **datos** que los **clientes no** necesitan **conocer**. También resulta especialmente útil cuando una clase contiene estructuras condicionales complejas con múltiples ramas para seleccionar comportamientos, ya que permite eliminarlas delegando la lógica en objetos estrategia.



Aparece la clase **Cliente**, que interactúa con la clase **GenSuperficies**. Esta clase actúa como contexto y define operaciones como especificarSalida() e interpolar(). En lugar de implementar directamente el algoritmo de cálculo, mantiene una referencia a una estrategia que sigue la interfaz IInterpolador. La interfaz **IInterpolador** declara el método calcular(), que encapsula el algoritmo que puede

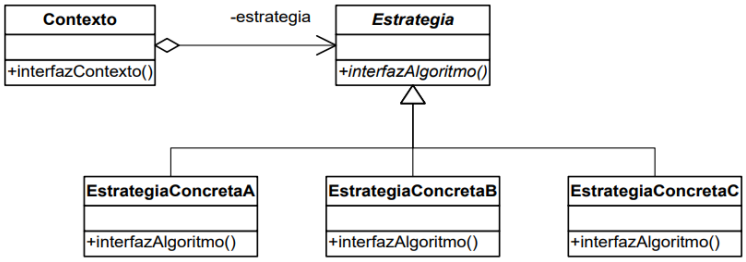
variar. Debajo se encuentran las clases **IDW**, **Spline** y **Kriging**, que son implementaciones concretas de distintas estrategias de interpolación. Cada una proporciona su propia versión del método calcular().

- <u>Participantes</u>

- **Estrategia** (Strategy): define la interfaz común que deben implementar todos los algoritmos. Esta interfaz es utilizada por el contexto para ejecutar el algoritmo seleccionado.

- **EstrategiasConcretas** (ConcreteStrategy): son las distintas implementaciones de esa interfaz, cada una con un algoritmo específico.

- **Contexto** (Context): es la clase que mantiene una referencia a una estrategia concreta y delega en ella la ejecución del algoritmo. Puede proporcionar una interfaz para que la estrategia acceda a los datos que necesita para operar.



- <u>Comportamiento</u>

El contexto y la estrategia colaboran para ejecutar el algoritmo seleccionado. Cuando el cliente realiza una petición al contexto, este **delega** la operación en la estrategia activa. El contexto puede **pasar** a la **estrategia** todos los **datos** necesarios para su ejecución o incluso pasarse a sí mismo como parámetro. Habitualmente, la estrategia concreta es **creada** por **otros objetos** (por ejemplo, el cliente) y se inyecta en el contexto, permitiendo configurar su comportamiento en tiempo de ejecución.

- <u>Ventajas e inconvenientes</u>

Definir múltiples **algoritmos intercambiables** ligados a diferentes soluciones de compromiso y modificar cualquiera de ellos de forma **independiente** al contexto. Además, elimina la necesidad de utilizar estructuras condicionales complejas. Sin embargo, **aumenta** el **número** de **clases** y **objetos** en el sistema. Asimismo, algunas estrategias simples pueden no utilizar toda la información que el contexto les proporciona, lo que puede resultar en cierta ineficiencia.

- <u>Implementación</u>

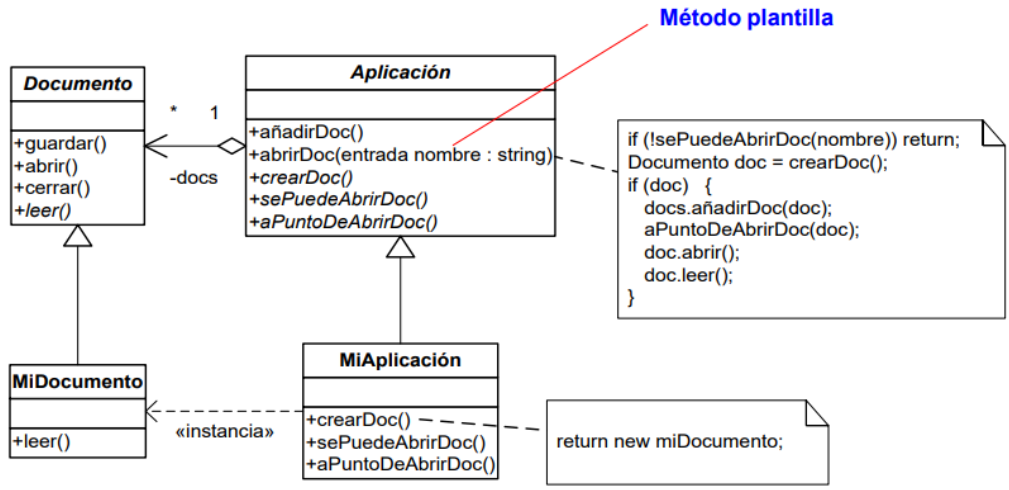
Una estrategia puede mantener una **referencia** al contexto si necesita acceder directamente a sus datos, lo que evita tener que pasar múltiples parámetros en cada llamada. Por otro lado, si el contexto **no** dispone de una **estrategia** asociada, puede **implementar** un comportamiento **por defecto** para garantizar el funcionamiento del sistema.

Template Method (Método Plantilla)

El patrón de diseño **Template Method** (método plantilla) pertenece a los patrones de comportamiento y tiene como objetivo definir el **esqueleto de un algoritmo** en una clase base, dejando algunos de sus pasos para que sean implementados o redefinidos por las subclases. Permite **redefinir ciertos pasos** del algoritmo sin cambiar su estructura. Cada paso del algoritmo puede invocar tanto operaciones **abstractas** (que deben ser implementadas por las subclases) como operaciones **ya definidas** en la clase base.

- <u>Aplicación</u>

Este patrón es especialmente útil cuando existen partes de un algoritmo que **no cambian** y otras que sí deben adaptarse a distintos contextos. Así, se evita la duplicación de código al **factorizar** el **comportamiento** común en una clase **abstracta**, mientras que las subclases se encargan de implementar el comportamiento específico. Además, permite controlar la extensión de las subclases mediante el uso de "**operaciones de enganche**", que son métodos definidos en la clase base, normalmente vacíos, que pueden ser redefinidos de forma opcional por las subclases.



A la izquierda se encuentra la clase **Documento**, que define operaciones básicas como guardar(), abrir(), cerrar() y leer(). Debajo aparece la clase **MiDocumento**, que hereda de Documento y redefine el comportamiento de alguna de estas operaciones, como leer(). En el centro del diagrama se encuentra la clase **Aplicación**, que contiene una colección de documentos y define el método

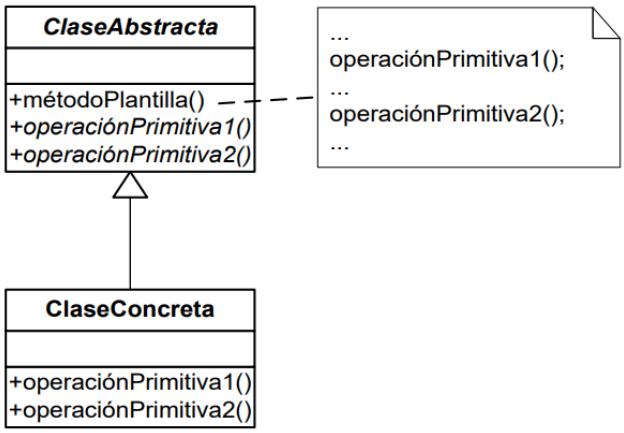
abrirDoc(nombre: string), que actúa como método plantilla. Este método establece el esqueleto del algoritmo: primero **comprueba** si se puede abrir el documento (sePuedeAbrirDoc()), luego **crea** el documento (crearDoc()), lo **añade** a la colección, **ejecuta** una operación previa (aPuntoDeAbrirDoc()), y finalmente **invoca** los métodos abrir() y leer() del documento.

Debajo de Aplicación aparece la clase **MiAplicación**, que hereda de ella e implementa los métodos abstractos como crearDoc(), sePuedeAbrirDoc() y aPuntoDeAbrirDoc(). De esta forma, la estructura del algoritmo permanece fija en la clase base, mientras que las subclases definen los detalles específicos.

- <u>Participantes</u>

- **Clase abstracta**: declara las **operaciones primitivas** (abstractas) que son definidas por las subclases para implementar los pasos de un algoritmo. También, define el método plantilla con el **esqueleto** del algoritmo.

- **ClaseConcretas**: que implementan dichas operaciones primitivas para realizar los pasos específicos.



- <u>Operaciones invocadas</u>

Dentro de las operaciones invocadas por el método plantilla, encontramos **operaciones primitivas** (abstractas en ClaseAbstracta y concretas en ClaseConcretas, obligatorias de redefinir), **operaciones concretas** (en ClaseAbstracta) ya implementadas en la clase base y **operaciones de enganche** (concretas en ClaseAbstracta), que son métodos vacíos en el padre y pueden ser redefinidos.

- <u>Comparación entre patrones</u>

Este patrón se relaciona con otros patrones de comportamiento que también buscan **encapsular** lo que varía dentro de un sistema. Por ejemplo, el patrón Strategy encapsula variaciones mediante composición en lugar de herencia. Asimismo, existen diferencias en la forma de comunicación entre objetos: mientras que Observer promueve una **comunicación distribuida** y débilmente

acoplada, Mediator **centraliza la interacción** en un único objeto, simplificando la comunicación. Otros patrones como Command, Observer o Mediator abordan el problema de **desacoplar emisores** y **receptores**, reduciendo las dependencias directas entre ellos.

En definitiva, los patrones de diseño, incluido Template Method, proporcionan un **vocabulario común** que eleva el nivel de abstracción y reduce la complejidad de los sistemas software. **Facilitan la documentación** de soluciones, ayudan en el **aprendizaje** de técnicas orientadas a objetos y **complementan** los métodos de diseño tradicionales al incorporar la experiencia acumulada. Además, favorecen la **transición** del análisis al diseño y guían la creación de software más **reutilizable**.